ChatGPT

Executive Summary

Zyven will evolve into a **production-grade Webhook Infrastructure Platform** that guarantees reliable delivery, visibility, and control for event-driven applications. It addresses the key pitfalls of native webhook handling (lost events, latency spikes, undeliverable messages) by inserting a durable queueing layer and rich observability. This PRD and roadmap lay out the target customers, core value, feature priorities, and system design needed to turn Zyven into a YC-ready solution.

We benchmark against existing solutions (Hookdeck, Stripe, Pipedream, Datadog, Sentry) and incorporate best practices (queue-first design, idempotent processing, retries with dead-letter queues, rate limiting, multi-region failover, robust observability)【11†L68-L71】【16†L266-L274】. The MVP focuses on safe ingestion and delivery with retries, HMAC signing, DLQ handling, and a basic dashboard. Subsequent releases add production features: SLA-grade uptime, per-tenant rate limits, circuit breakers, priority queues, advanced analytics (latency/error metrics, dashboards), SDKs/CLI, Terraform provider, and full multi-region reliability. By Month 6 Zyven should be demonstrably capable of handling millions of webhooks with 99.9%+ uptime and comprehensive SRE tooling【30†L333-L341】【16†L386-L388】.

Target Users and Personas

- **Backend Engineers at SaaS/API companies:** Developers building event-driven services (e.g. payments, e-commerce, IoT) who need *guaranteed* webhook delivery and the ability to trace and replay events. They value reliability (no lost events), observability (failed payloads, latencies), and easy integration.
- **Platform/SRE Teams:** Operations or site-reliability engineers responsible for uptime and compliance. They need **multi-tenant isolation**, high SLA, multi-region failover, and auditing. They want metrics, alerts on failures/backlogs, and tools (CLI/DSL/SDK) to debug issues.
- **Startups/High-Growth Firms:** Companies rapidly integrating many third-party services (e.g. Stripe, GitHub, PagerDuty). They often outgrow DIY solutions and need turnkey reliability and scaling without reinvesting engineering time【16†L339-L347】.

Example Persona: "As a payments platform engineer, I want to queue and retry Stripe webhooks off my main API so that I never miss a transaction event even if my servers go down. I also need dashboards to quickly diagnose failures."

Core Value Proposition

Zyven provides **"drop-in" webhook durability and observability** so teams can decouple event delivery from their core APIs. It ensures **at-least-once delivery** (no silent data loss) via durable storage and intelligent retries【11†L68-L71】. It also offers **full visibility** and control: search and replay of events, payload logs, delivery metrics, and alerting【16†L317-L325】. In short, it turns webhooks into a reliable,

monitorable data stream, offloading complex distributed-systems concerns (backoff, circuit-breaking, multi-tenancy, SLA) to a specialized platform【11†L68-L71】【16†L266-L274】.

- **Guaranteed Delivery:** Durable "outbox" storage (PostgreSQL) at ingest ensures no event is lost, plus automated retries with backoff and DLQs【11†L68-L71】【16†L266-L274】.
- **High Scalability & SLA:** Elastic, horizontal scaling and multi-region redundancy aim for ≥99.9% uptime (target 99.99%+ for paid tiers)【30†L333-L341】【16†L386-L388】.
- **Observability & Control:** Metrics (throughput, latencies, error rates) and a rich dashboard (search, filters, graphs) give full audit trail and one-click replay【16†L317-L325】【13†L215-L219】.
- **Safety & Security:** Built-in HMAC signing/verification, idempotent ingestion, SSRF protections, and per-tenant isolation align with best security practices【8†L298-L304】【16†L266-L274】.

Competitor Benchmark & Feature Gap

Feature	Datadog	Sentry	Hookdeck / PostHog	Pipedream	Stripe Webhooks
Reliable delivery (at-least-once)	– (monitoring tool)	– (error tool)	Yes【16†L261-L269】【16†L363-L371】	Yes (event ingestion)	Yes (72h retries)【18†L3-L7】
Durable storage (outbox)	–	–	Yes (ACK + DB)【16†L248-L251】	(workflow logs)	Partial (event log API)
Exponential retry + jitter	No	No	Yes【16†L264-L274】	(basic retry)	No
Dead-letter queue	No	No	Yes【11†L68-L71】	No	No
Rate limiting / backpressure	No	No	Yes (configurable)【16†L270-L274】	Limited	No
Multi-region failover	(cloud infra)	(cloud infra)	Yes (auto failover)【16†L288-L293】	(hosted)	(Stripe global infra)
Event search & replay	No	No	Yes (search, replay)【16†L319-L322】	Limited	Yes (via API calls)
HMAC signing / validation	No	No	Yes (HMAC-SHA256)【8†L298-L304】	(can custom)	Yes (official)
CLI / SDK / Terraform	API / no CLI	CLI & API	CLI + Terraform【16†L381-L388】	CLI	Stripe CLI
Observability (metrics, logs)	Yes (metrics/ APM)	Yes (error logs)	Yes (latency, throughput)【13†L215-L219】	Yes (workflow UI)	Basic (dashboard UI)

Feature	Datadog	Sentry	Hookdeck / PostHog	Pipedream	Stripe Webhooks
SLA / Uptime guarantee	~99.9% (max)	~99.9%	99.999% (financially-backed) 【16†L386-L388】	None (no SLA)	N/A (payment provider)

Key takeaways: Hookdeck is the closest feature match – it offers persistent ingestion, smart retries, DLQs, rate-limits and search/replay【11†L68-L71】【16†L317-L322】. Datadog/Sentry only monitor data but do not provide webhook delivery guarantees. Pipedream handles flows and history but lacks DLQs or advanced retries. Stripe's built-in webhooks auto-retry for ~72h【18†L3-L7】 but require manual handling (replay via API) and don't isolate tenants or provide dashboards. Zyven's goal is to match or exceed Hookdeck/ Pipedream with an open, highly-engineered solution geared toward YC-level startups.

Product Requirements

Personas and Use Cases

1. **API Developer (Alice):** Needs a way to offload webhook delivery so her core service always responds fast. She wants an easy API to register endpoints, send events, and query status. **Use case:** Integrate Zyven behind her API to enqueue events; use the dashboard/CLI to inspect failures.
2. **SRE / Platform Engineer (Bob):** Manages high-scale event ingestion. Needs SLAs, alerts, and robust backpressure controls. **Use case:** Configure per-tenant rate limits and concurrency caps; monitor queue depths/latencies; handle failover across regions.
3. **Startup CTO (Carol):** Fast time-to-market, minimal ops overhead. Wants a SaaS-like onboarding (signup, API key, CLI). **Use case:** Use Zyven free tier to begin ingesting Stripe/Github webhooks in minutes, then upgrade for higher throughput and support.

Core (MVP) Features

(Release 0: Zapier Automation-style Reliability)

- **Event Ingestion API:**
- *Description:* A REST endpoint to receive webhook POSTs from clients or third-party providers. It immediately ACKs and writes the event to Postgres (the **reliability boundary**)【16†L248-L251】.
- *User Story:* As a developer, when my API generates an event, I can `POST /v1/events` to Zyven with my payload, and Zyven responds quickly (200 OK).
- *Acceptance:*
  - Returns 200 if event format is valid, otherwise 4xx.
  - Persists event to DB and enqueues a delivery job in the same transaction (ensuring durability).
  - Generates a unique event ID.
- **Idempotency & Deduplication:**

- *Description:* Ensure retries of the same request do not double-process. Clients supply an `Idempotency-Key` header or Zyven generates it.
- *User Story:* As a client, if I resend a webhook with the same idempotency key, Zyven recognizes it and does not create a new event.

- *Acceptance:*

    - On duplicate key, respond 200 with original event ID (no duplicate event in DB).
    - The DB has a unique index on the idempotency key.

- **Delivery Engine (Worker + Queue):**

- *Description:* A BullMQ-backed job queue picks up events and sends them to the configured external URL (destination). Workers apply exponential backoff on failures.
- *User Story:* As a platform engineer, if the destination HTTP returns 500 or times out, Zyven retries with increasing delays until success or max retries.

- *Acceptance:*

    - After a failed attempt (non-2xx or timeout), schedule a retry with jitter (e.g. 2^n minutes) and increment retry count.
    - After X failed retries, mark event as **DEAD_LETTERED**.
    - On success (2xx), mark event **DELIVERED** and stop retries.

- **Dead-Letter Queue & Replay:**

- *Description:* Permanently failed events (e.g. 4xx or after max retries) land in a DLQ. The UI/CLI allows listing DLQ events and re-queuing them.
- *User Story:* As a developer, I can see which events failed permanently and press “Requeue” to try again after fixing the issue.

- *Acceptance:*

    - DLQ events flagged in DB (status=D”DEAD_LETTERED”).
    - API/UI shows DLQ items with timestamps and error codes.
    - “Replay” operation resets status and schedules a new attempt.

- **HMAC Signing & Verification:**

- *Description:* Outbound requests are signed (HMAC-SHA256) using the secret per endpoint【8†L298-L304】; destination can verify. Incoming events may also be verified.
- *User Story:* As an admin, I configure a secret for my endpoint. All callbacks from Zyven include an `X-Signature` header using that secret.

- *Acceptance:*

    - On delivery, compute HMAC of payload, include in header.
    - Provide a utility or docs so receiver can validate.

- **Security & Compliance:**

- Input sanitization on ingestion (e.g. JSON schema validation).
- SSRF prevention: Reject outbound requests to private IPs or disallowed domains【8†L298-L304】 (use an outgoing proxy with IP whitelisting).
- Store secrets encrypted (e.g. AWS KMS).

- Compliance: Aim for SOC2 readiness (Hookdeck is SOC2 compliant【13†L61-L68】), encryption at rest in Postgres, and encrypted connections.

- **Basic Dashboard & Metrics:**

- *Description:* Web UI to see event list, statuses (RECEIVED, DISPATCHING, RETRY_SCHEDULED, DELIVERED, DEAD_LETTERED) and attempt logs (timestamps, status codes, latency).
- *User Story:* As an engineer, I log into Zyven and see my last 100 events, filter by status, and inspect their delivery logs.

- *Acceptance:*

    - List view: columns (ID, endpoint, created, status, retries).
    - Detail view: show full payload, all delivery attempt logs (timestamps, code, response).
    - Expose simple metrics (total events, delivered, failed) via the UI.

- **API Design (MVP):**

- Endpoints:
    - `POST /v1/events` – Create event (ingest).
    - `GET /v1/events/{id}` – Get event status/logs.
    - `POST /v1/endpoints` – Create outbound endpoint config (URL, secret, headers, rate limit).
    - `GET /v1/endpoints` – List endpoints (for a tenant).
    - `POST /v1/endpoints/{id}/replay` – Replay DLQ events.
    - Auth via API key (bearer token) for all management calls.

- Use REST JSON; GraphQL could be added later.

- **Tenancy & Access Control:**

- Each customer (“tenant”) has isolated data (events/endpoints) under their account.
- API keys scoped per tenant.
- Future: Roles (admin/developer) and teams.

Features for Next Releases

(After MVP – Release 1 to 3 prioritized by value)

- **Release 1 (Scale & Safety):**
- *Rate Limiting*: Configurable qps per endpoint/tenant to protect downstream (e.g. 5/sec for legacy services)【16†L270-L274】. Implement token-bucket algorithm.

- *Concurrency Control*: Max parallel deliveries per tenant.
- *Priority Queues*: Allow premium vs standard tiers (e.g. express queue for paid customers).
- *Circuit Breaker*: If an endpoint fails repeatedly (high 5xx), temporarily pause retries (open circuit) for a period to avoid queue pileup.
- *Multi-Zone Deployment*: Ability to run workers across regions; use DNS failover or a global load balancer (e.g. AWS Route 53) for ingestion.

- *API Enhancements*: Webhooks schema versioning support (v1, v2 payload formats); signature rotation (allow multiple active keys) to support secret updates.

- **Release 2 (Observability & Developer Experience):**

- *Metrics & Dashboards*: Export Prometheus metrics (event rates, latencies, queue lengths, retry counts). Pre-built Grafana dashboards.
- *Distributed Tracing*: Instrument with OpenTelemetry to trace a webhook event from ingest through delivery.
- *Alerting & SRE Playbook*: Define out-of-box alerts (e.g. >5% failure rate, queue backlog growth). Provide incident runbooks (e.g. "Worker crashed: restart service; if backlog grows, scale out workers or check DB connections").
- *SDKs & CLI*: Provide client libraries (Node, Python, Go) for easy integration (wrappers to call Zyven API and verify signatures). CLI tool to inspect events (`zyven events list`), trigger replays, or tail live logs.

- *Terraform Provider*: Allow infrastructure as code setup of endpoints, quotas, etc.

- **Release 3 (Enterprise-grade):**

- *Geo-Redundancy*: Active-active data layer (multi-master DB or geo-replicated Postgres). Automatic failover across regions.
- *Custom SLAs*: Options for 99.9% vs 99.99% uptime with financial credit (like Hookdeck does) 【16†L386-L388】.
- *Advanced Security*: Tenant-specific IP whitelisting, VPC peering, Web Application Firewall integration. SOC2/HIPAA compliance documentation.
- *More Destinations*: Built-in connectors (e.g. send to AWS SQS, PubSub, SNS, or email) as alternative to HTTP endpoints.
- *Billing/Onboarding UI*: Usage dashboards, team/user management, plan upgrades.

Each feature includes **acceptance criteria** defined above (MVP) or in detailed stories in product backlog.

User Stories (Examples)

- *As a backend developer*, I want Zyven to **immediately ACK** my incoming webhook request and store it reliably, so that my core API doesn't block and no events are lost.

- **AC:** Zyven returns 200 OK within 50ms of request; event is persisted before responding.

- *As a developer*, I want to set an **Idempotency Key** on requests, so that accidental double-sends don't create duplicates.

- **AC:** Using the same key twice results in one stored event; second call returns existing event ID.

- *As a dev*, I want Zyven to **retry deliveries** on failure with exponential backoff, so I don't have to code retry logic.

- **AC:** Non-2xx responses trigger a retry after increasing intervals (e.g. 10s, 30s, 90s…); after max 5 attempts, event status = `DEAD_LETTERED`.

- *As an SRE*, I want **rate limiting**, so one noisy tenant can't overwhelm my service.

- **AC:** If 100 requests come in within one second but limit = 10/s, Zyven queues/delays excess and delivers at 10/s.

- *As an SRE*, I want to be **notified of failures**, so I can respond to problems quickly.

- **AC:** If >X events fail in a minute, send alert (e.g. email/Slack). Dashboard shows failure spike in real-time.

Non-Functional Requirements

- **Availability/SLA:** Target 99.9% uptime (≤43.2 minutes downtime/mo) for standard tier, with enterprise option ≥99.99%【30†L333-L341】【30†L346-L354】. Use redundant servers, health checks, and autoscaling.
- **Durability (RPO/RTO):** Aim for **RPO=0** (no lost events after DB commit). Use database replication (e.g. PostgreSQL synchronous standby) for zero data loss. Aim RTO (recovery) within ~5 minutes of failure by fast restart or failover procedures.
- **Performance (Throughput/Latency):** MVP: handle ~10K events/sec (configurable), end-to-end delivery latency target <500ms per event (with retries adding minimal overhead). Scale to 100K+/sec via horizontal sharding.
- **Scalability:** Support auto-scaling of ingestion and worker processes. Stateless API nodes behind load balancer; queue cluster (Redis or Kafka) that can scale. Multi-region: replicate DB or use a global datastore (e.g. CockroachDB / Aurora Global) for cross-region.
- **Multi-tenancy:** Strong tenant isolation – no data bleed. Per-tenant rate/usage quotas. Fair scheduling so one customer (tenant) can't consume 100% of workers.
- **Security:** API authentication (token keys, OAuth2 eventually). Encrypt all secrets at rest. Outbound SSRF defense via proxy filtering【8†L298-L304】. Follow OWASP best practices (https only, input validation). Role-based access control in UI. Audit logging of all config changes.
- **Compliance:** Build towards SOC2 Type II (secure development practices, monitoring), GDPR (data export/deletion), HIPAA (if targeting health).

System Architecture

We adopt a **queue-first, event-driven** architecture with a transactional outbox pattern【11†L68-L71】.
Below is the high-level flow:

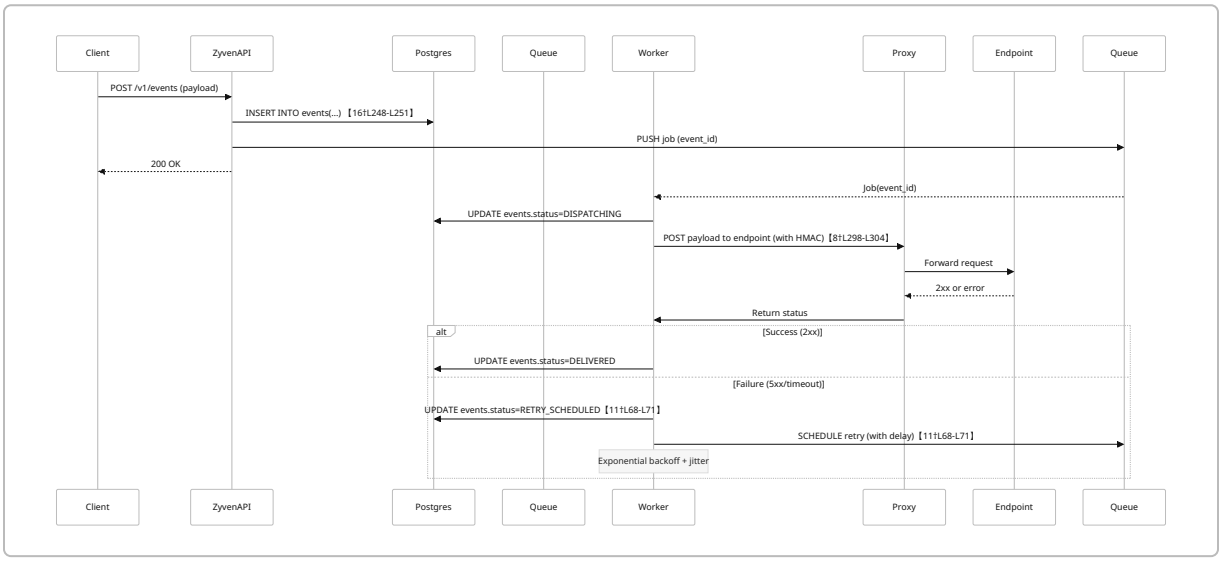

Figure: *Webhook event flow through Zyven system (transactional outbox ensures durability【16†L248-L251】).*

An entity-relationship diagram of core tables:

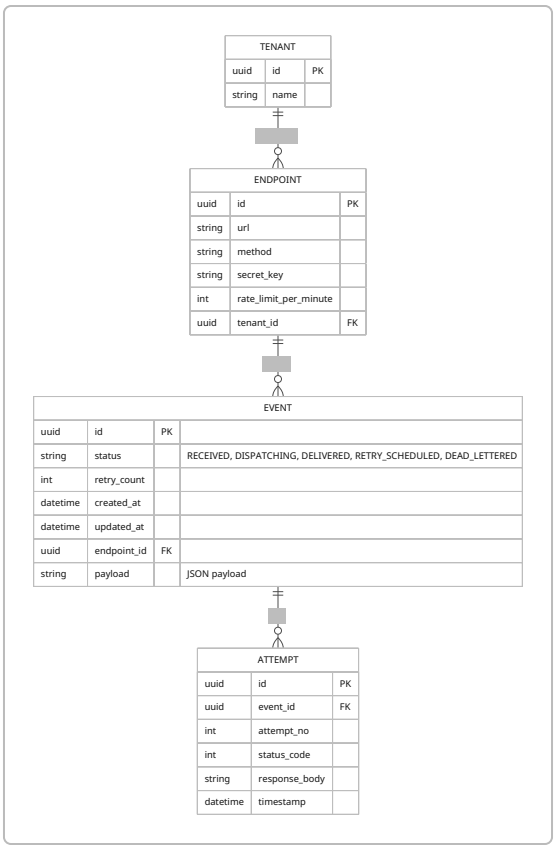

Figure: *ER model for tenants, endpoints, events, and delivery attempts.*

API Design

We use RESTful JSON APIs with API-key (bearer token) authentication. (GraphQL could be added later.) All endpoints require the client's API key.

Major Endpoints:

- `POST /v1/events` – Ingest an event. Body: `{ "endpoint_id": "...", "payload": {…}, "idempotency_key": "abc" }`. Returns `{event_id}`. Idempotency key deduplication.
- `GET /v1/events/{event_id}` – Get event status and attempt log.
- `GET /v1/events?status=RETRY_SCHEDULED` – List of events by status (with pagination).
- `POST /v1/endpoints` – Create an endpoint config: `{ url, method, secret_key, max_retries, retry_schedule, rate_limit }`.
- `GET /v1/endpoints` – List configured endpoints for the tenant.
- `GET /v1/endpoints/{id}/events` – List events for that endpoint.
- `POST /v1/events/{event_id}/replay` – Replay a DLQ'd event (reset status to RECEIVED).
- `POST /v1/webhook-configs/{endpoint_id}/rotate-secret` – Rotate HMAC secret (maintain old key for transition).
- `POST /v1/tenants/{id}/limit` – (Admin) set tenant-wide rate/concurrency limits.

Webhooks Schema: Customizable per client. Zyven does not enforce a fixed schema, but supports JSON validation rules on payload if configured. An alternative design is to offer GraphQL: not planned initially.

Data Model & Persistence

- **Transactional Outbox:** In the ingest API handler, insert the event into Postgres and publish to BullMQ within the same DB transaction (or using an *outbox* table pattern). This ensures once the API responds, the event is durably logged【16†L248-L251】.
- **Tables:** Core tables include `tenants`, `endpoints`, `events`, `attempts`, `audit_logs`. (ERD above.) Add indexes on foreign keys, and on `events.status` for quick querying of pending items【8†L300-L304】.
- **Outbox Pattern:** Alternatively, an explicit `outbox` table can hold pending events; a background thread moves them to the queue. This is standard for guaranteeing atomicity of DB write + message publish【11†L68-L71】.
- **Dead-letter:** Failed events can be stored in an `events_dead` table or marked in `events.status` with a flag. The replay UI will simply re-insert or re-activate them.
- **Archival:** For long-term storage, older events (>X days) could be archived to cold storage or summarized, to keep production DB small.

Queueing Strategy

Zyven must support high throughput with durability. Options:

- **BullMQ (Redis Streams):** Simple setup, Node.js native. Pros: Easy to integrate (already used), quick start, rich retry/delay features. Cons: Relies on Redis (needs persistence/rdb/aof), single-master (Failover possible with Redis Cluster). Best for moderate scale and developer productivity.
- **Kafka (or Redpanda):** If expected volume grows to millions/sec, Kafka offers true distributed log with tunable retention and consumer groups. Pros: Handles trillions of messages【25†L58-L66】, built-in fault tolerance. Cons: Operationally complex, long setup (as Hookdeck notes【16†L339-L344】).
- **Redis Streams (standalone):** Can be used directly or via BullMQ. Similar trade-offs to Kafka on simpler scale. Dragonfly's comparison notes Kafka's advantages at web-scale【25†L53-L61】.

Recommendation: Start with **BullMQ/Redis** for MVP (simpler, already in code). Design code so that the queue abstraction could switch to Kafka later. Document the tradeoff: for now, a single reliable Redis node (or small cluster) suffices. Should use AOF persistence and replication for durability.

Migration Plan: If/when reaching Redis limits, consider migrating to Kafka/Redpanda. Consumers code would mostly switch queue client. Use schema definitions so that queue items are simple event IDs (we already persist content in DB).

Reliability Features

To match YC/platform expectations, include these:

- **Rate Limiting & Backpressure:** Cap per-endpoint/per-tenant throughput. Unaccepted bursts are *queued*, not dropped【13†L133-L141】. Implement rate-limiter in worker: e.g. token bucket on dispatch, or let BullMQ "delay" if needed.
- **Circuit Breaker:** Track consecutive failures per destination. If threshold exceeded, trip the circuit: temporarily stop attempting further deliveries (perhaps pause retries for a minute) to avoid endless failures. This protects the queue from backing up.
- **Priority Queues:** Maintain separate queues or priorities (BullMQ supports priorities). Allocate higher CPU/throughput to paid customers.
- **Retries & Backoff:** Exponential backoff with full jitter for retries, per [Hookdeck best practices]【16†L264-L274】. Make schedules configurable per endpoint (some may prefer fixed delays or more retries).
- **Poison Messages Handling:** If an event fails repeatedly due to bad payload, mark dead-letter. Prevent infinite retry loops. (Seen in [11]: DLQs prevent lost events【11†L68-L71】.)
- **Exactly-Once vs At-Least-Once:** Clearly document that Zyven guarantees *at-least-once* delivery. Encourage idempotency on receivers. This aligns with distributed systems principles【11†L132-L136】.

Observability & SRE

- **Metrics:** Expose Prometheus metrics for: events/sec (ingest, delivered, failed), queue length, delivery latencies (p50, p95, p99), retry counts, dead-letter counts, HTTP error codes. Hookdeck metrics example: latency, throughput, error rates, backpressure【13†L215-L219】.
- **Logging:** Structured logs (JSON) for ingestion and worker processes. Each delivery attempt log should include event ID, status, response code.
- **Distributed Tracing:** (Planned) Use OpenTelemetry to correlate an event's lifecycle across services (API, workers).
- **Dashboard:** A web UI for key dashboards (e.g. time series of throughput and failures). We can embed Hookdeck's dashboard concept; it shows per-source metrics.
- **Alerts & Runbook:** Define alert triggers, e.g.:
- **High failure rate:** e.g. if >5% of events in 5 min fail (configurable).
- **Growing queue:** if backlog > threshold for 1min.
- **Worker health:** worker down/exception logs.
- **Action:** e.g. "If failures spike, verify destination endpoints; use UI to inspect payload errors; if queue backlog grows, scale out workers or investigate DB latency." Provide these in a runbook document.
- **Incident playbook examples:** On Redis or DB outage: fail open (API writes will error; an SRE should restart). On full-queue: throttle producers if possible.

System Architecture Diagrams

Below is a **sequence diagram** of Zyven's processing flow (see above code block).

Figure: Event flow (ingest to delivery).

We can also visualize a **deployment architecture** (in text):

```
graph LR
  Client-->API[Zyven API Servers]
  API-->DB[(PostgreSQL DB)]
  API-->Queue[Redis (BullMQ)]
  Queue-->Workers[Worker Pool]
  Workers-->Proxy[Outgoing Proxy Cluster]
  Proxy-->Endpoint[Customer HTTP Endpoint]
  Subgraph Observability
    API-->Metrics[Prometheus/Grafana]
    Workers-->Tracing[OpenTelemetry]
    DB-->Metrics
  end
  Subgraph Global
    AWSUS[AWS US-East-1] -.-> AWSEU[AWS EU-West-1]
    AWSUS -.-> AWSAP[AWS AP-South-1]
    DB---|Replication| AWSUS
    DB---|Read Replica| AWSEU
    Queue---|Cluster| AWSUS
    Queue---|Cluster| AWSEU
  end
```

Deployment & Infrastructure

- **Containerization:** All services (API, workers, proxy) run in Docker containers. Use Kubernetes or ECS/EKS for orchestration. Keep containers stateless (externalize config/secrets).
- **CI/CD:** GitHub Actions to build/test and push images on commit. Automated deploy to staging/prod.
- **Infrastructure as Code:** Terraform for provisioning AWS (or chosen cloud):
- VPC, subnets (multi-AZ), RDS (Postgres) with multi-AZ, ElastiCache (Redis Cluster) or MSK (Kafka), ALB for API, Autoscaling Groups/ECS tasks.
- Use managed services (RDS, ElastiCache/Kafka) for reliability.
- Include TLS cert provisioning (ACM) and DNS.

- Terraform module for Zyven resources (endpoints, users).

- **Multi-Region:** Start with one region (e.g. AWS us-east). Plan for cross-region: either active-active (replicating DB, use global LB) or active-passive failover.

- **Secrets Management:** Use AWS KMS or HashiCorp Vault for endpoint secrets and API keys. Rotate keys regularly.

- **SSRF Proxy:** Implement a small service that the worker calls, which then makes the outbound HTTP. The proxy inspects DNS/IP and blocks disallowed targets (like 169.254.x.x metadata)【8†L298-L304】. Deploy proxies in same private networks.

Security

- **Authentication:** API keys (JWTs or opaque tokens). Optionally OAuth for UI.
- **Network:** APIs only over HTTPS. Outbound proxy restricts IP ranges (SSRF protection as Zyven design uses)【8†L298-L304】. Use private subnets for databases.
- **Secrets:** Store endpoint secrets encrypted (KMS), never log secrets. Support secret rotation as a feature.
- **Tenant Isolation:** Strict row-level security or per-tenant schemas so one tenant cannot read another's data.
- **Audit Logs:** All admin actions (creating endpoints, changing config) are logged in an audit table. UI shows who and when did what.
- **Compliance:** Prepare documentation for SOC2 (change management, incident response, logging) and GDPR (data deletion on request).

Testing Strategy

- **Unit & Integration Tests:** For all code paths (ingest API, worker logic).
- **Load Testing:** Use a tool like K6 or Gatling to simulate webhook spikes (e.g. 10K/s) and measure throughput, latency, and queue behavior.
- **Chaos Testing:** Simulate worker crashes, Redis failover, DB outage. Verify system recovers (tests can be scripted).
- **Chaos for SSRF:** Ensure proxy blocks private IPs by test-calling a known restricted address.
- **End-to-End:** Use test webhooks (e.g. Stripe's test mode or a mock service) to verify retries and replay.
- **Compliance Tests:** Penetration test focused on SSRF, injection, auth.

Roadmap & Milestones

A **6-month plan** (Phase 1–3) is outlined below, with weekly goals.

Month/Week	Milestone / Deliverables	Key Tasks & Skills
Month 1	**Core Prototype:** Ingest + deliver, basic UI.	Implement POST /events, DB schema, BullMQ worker, retry logic, HMAC signing. Simple Next.js dashboard: event list/logs. (Skills: Node.js, Redis, SQL, Docker.)
W1: Setup infra and tools (Docker, TS infra).	Setup Postgres, Redis; stubs for API and worker.	

Month/Week	Milestone / Deliverables	Key Tasks & Skills
W2: Ingestion API + DB outbox	Code /events endpoint, transactional write, idempotency.	
W3: Worker + retry/ deliver	Implement BullMQ consumer, HMAC signing, status updates.	
W4: Dashboard UI basics	List events/status, show attempts.	
Month 2	**Resilience & Scale:** Rate limits, Circuits.	Add rate-limiting logic, configurable per-endpoint. Circuit breaker prototype. Metrics collection.
W5: Per-tenant config & limits	UI forms for rate limits; enforce in worker.	
W6: Circuit breaker logic	Detect high failures, pause retries.	
W7: Retry enhancements	Add jitter, customizable schedules.	
W8: Write first integration tests (load simulators).	Test high-error scenarios.	
Month 3	**Observability:** Logging, metrics, alerts.	Integrate Prometheus + Grafana, instrument code with metrics (e.g. `incr events_total`, `hist latency`). Set up alert rules (e.g. on failure rate).
W9: Metrics & Monitoring	Expose metrics, build dashboards (Grafana).	
W10: Alerting / SRE docs	Define alerts (via CloudWatch/Grafana). Document incident playbook.	
W11: Distributed tracing (optional)	Add OpenTelemetry spans across API & worker.	
W12: End-to-end QA & docs	Verify resilience, write deployment README.	
Month 4	**Developer UX:** SDKs, CLI, Terraform.	Build and publish Zyven SDKs (npm/pypi packages). CLI tool (`zyven`), Terraform provider skeleton. API key management.

Month/Week	Milestone / Deliverables	Key Tasks & Skills
W13: SDK v1 (Node.js)	Publish npm package for Zyven client.	
W14: CLI + auth management	CLI to list/replay events. Manage API keys.	
W15: Terraform provider (alpha)	Define resources for endpoints, quotas.	
W16: User roles & teams (UI)	Add admin users, token scopes.	
Month 5	**Enterprise Features:** Multi-region + SLAs.	Deploy DR (multi-AZ/Postgres). Implement failover. Prepare SLA docs (99.9/99.99%) and start hook up monitoring for SLA.
W17: Multi-AZ DB & cache	Configure RDS Multi-AZ, Redis cluster.	
W18: Geo-failover plan	Test DB failover, route53 DNS failover.	
W19: SLAs & terms	Define uptime guarantees, downtime SLAs.	
W20: Compliance prep	Audit code/security; start SOC2 checklist.	
Month 6	**Polish & Launch:** Documentation + marketing.	Finalize docs, create demo videos, publish blog on webhook best practices. Start user beta testing. Scale outreach.
W21: Final documentation	Complete README, API docs, runbooks.	
W22: Tech blog (architecture, case study)	Write blog posts (e.g. “How Zyven scales to 100M events”).	
W23: Beta testing & feedback	Invite early users, fix issues.	
W24: Launch & apply to YC startups	Prepare pitch deck; apply to YC, contact early adopters.	

Resource estimate: One full-stack developer (Node/TS) and one DevOps engineer (cloud/Infra) would ideally execute this plan. No hard budget limits (favor managed services to save ops time).

Success Metrics

- **Reliability:** <1 lost event per million (aim 0); 99.9%+ delivery success rate in normal ops.
- **Performance:** E2E latency (ingest to first delivery attempt) <500ms at median; system sustains target throughput (e.g. 10k eps) with <5% CPU usage.
- **Scalability:** Demonstrated handling of bursts 2× expected load without human intervention.
- **Adoption:** Growth in paid users, number of events processed.
- **Developer Happiness:** Internal metric: >90% of webhook events delivered successfully on first attempt (reducing manual replays).

Competitor Gap Analysis

Comparing feature sets (see table above), Zyven's roadmap closes gaps by adding what Hookdeck/Pipedream offer and more: **enterprise SLAs, multi-region, CLI/SDK, Terraform provider, team management, compliance**. For example, while Stripe lets you manually list and replay missed events via API【18†L3-L7】, Zyven automates retries and DLQs end-to-end with a UI. Similarly, unlike Datadog or Sentry (which only observe), Zyven actively *controls* webhook delivery.

In summary, by end of development Zyven will achieve parity or superiority in all key categories of webhook reliability (DLQ, backoff, signing, metrics) and add advanced capabilities (geographic redundancy, fine-grained rate limits, automated recovery, rich developer tools) that impress both YC founders and enterprise SRE teams.
